

SKILL WEEK 11

Task 1: Module Integration

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components

To divide functionality into separate C modules, connect them through a header file, and test their end-to-end execution.

Program

Create three files: main.c, calculator.c, and calculator.h.

File 1: calculator.h

```
#ifndef CALCULATOR_H
#define CALCULATOR_H

int add(int a, int b);
int multiply(int a, int b);

#endif
```

File 2: calculator.c

```
#include "calculator.h"

int add(int a, int b) {
    return a + b;
}

int multiply(int a, int b) {
    return a * b;
}
```

File 3: main.c

```
#include <stdio.h>
#include "calculator.h"

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("Addition: %d\n", add(a, b));
    printf("Multiplication: %d\n", multiply(a, b));

    return 0;
}
```

Output

```
amrutha@blue:~$ mkdir ~/SKILL_Week11
amrutha@blue:~$ cd ~/SKILL_Week11
amrutha@blue:~/SKILL_Week11$ nano calculator.h
amrutha@blue:~/SKILL_Week11$ nano calculator.c
amrutha@blue:~/SKILL_Week11$ nano main.c
amrutha@blue:~/SKILL_Week11$ gcc main.c calculator.c -o integrated
amrutha@blue:~/SKILL_Week11$ gcc main.c calculator.c -o integrated
amrutha@blue:~/SKILL_Week11$ ./integrated
Enter two numbers: 5 4
Addition: 9
Multiplication: 20
```

Observation

The calculator functionality was separated into a module and connected to the main program through a header file. The modules compiled and executed together successfully.

Result

Module integration and end-to-end execution were successfully implemented.

Task 2: Runtime Error Handling

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios

To detect invalid input and runtime errors, display meaningful error messages, and allow the program to terminate gracefully.

Program

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    int a, b;
    FILE *log;

    log = fopen("error.log", "a");

    if (log == NULL) {
        perror("Unable to open log file");
        return 1;
    }

    printf("Enter two integers: ");

    if (scanf("%d %d", &a, &b) != 2) {
        printf("Error: Invalid input. Please enter integers.\n");

        fprintf(log, "ERROR: Invalid input detected.\n");

        fclose(log);
        return 1;
    }
}
```

```

    if (b == 0) {
        printf("Error: Division by zero is not allowed.\n");

        fprintf(log, "ERROR: Division by zero attempted.\n");

        fclose(log);
        return 1;
    }

    printf("Result: %d / %d = %.2f\n",
           a, b, (float)a / b);

    fprintf(log, "INFO: Division completed successfully.\n");

    fclose(log);

    return 0;
}

```

Terminal

```

amrutha@blue:~/SKILL_Week11$ nano error_handling.c
amrutha@blue:~/SKILL_Week11$ gcc error_handling.c -o error_handling
amrutha@blue:~/SKILL_Week11$ ./error_handling
Enter two integers: 20 5
Result: 20 / 5 = 4.00
amrutha@blue:~/SKILL_Week11$ ./error_handling
Enter two integers: 20 0
Error: Division by zero is not allowed.
amrutha@blue:~/SKILL_Week11$ ./error_handling
Enter two integers: hello world
Error: Invalid input. Please enter integers.
amrutha@blue:~/SKILL_Week11$ cat error.log
ERROR: Invalid input detected.
INFO: Division completed successfully.
ERROR: Division by zero attempted.
ERROR: Invalid input detected.

```

Input 1 — Valid Input

20 5

Output

amrutha@blue:~/SKILL_Week11\$./error_handling

Enter two integers: 20 5

Result: 20 / 5 = 4.00

Input 2 — Division by Zero

Run again:

```
./error_handling
```

Input:

```
20 0
```

Output:

```
Enter two integers: 20 0
```

```
Error: Division by zero is not allowed.
```

Input 3 — Invalid Input

```
./error_handling
```

Input:

```
hello world
```

Output:

```
Enter two integers: hello world
```

```
Error: Invalid input. Please enter integers.
```

Verify Error Log

```
cat error.log
```

Example:

```
INFO: Division completed successfully.
```

```
ERROR: Division by zero attempted.
```

```
ERROR: Invalid input detected.
```

Observation

Invalid input and division-by-zero conditions were detected. Appropriate error messages were displayed and failures were recorded in error.log.

Result

Runtime error detection, logging, and graceful error handling were successfully implemented.

Task 3: Valgrind / Memory Debugging

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings

To allocate dynamic memory, detect memory leaks using Valgrind, release the allocated memory, and verify that no memory leaks remain.

Program

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *numbers;

    numbers = malloc(5 * sizeof(int));

    if (numbers == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        numbers[i] = (i + 1) * 10;
    }

    printf("Allocated numbers:\n");

    for (int i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }

    printf("\n");

    free(numbers);
    numbers = NULL;

    printf("Memory released successfully.\n");

    return 0;
}
```

Output

```
amrutha@blue:~/SKILL_Week11$ nano memory.c
amrutha@blue:~/SKILL_Week11$ gcc -g memory.c -o memory
amrutha@blue:~/SKILL_Week11$ ./memory
Allocated numbers:
10 20 30 40 50
Memory released successfully.
amrutha@blue:~/SKILL_Week11$ valgrind --version
valgrind-3.26.0
amrutha@blue:~/SKILL_Week11$ valgrind --leak-check=full ./memory
==4473== Memcheck, a memory error detector
==4473== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==4473== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==4473== Command: ./memory
==4473==
Allocated numbers:
10 20 30 40 50
Memory released successfully.
==4473==
==4473== HEAP SUMMARY:
==4473==   in use at exit: 0 bytes in 0 blocks
==4473==   total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==4473==
==4473== All heap blocks were freed -- no leaks are possible
==4473==
==4473== For lists of detected and suppressed errors, rerun with: -s
==4473== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Observation

Dynamic memory was allocated using `malloc()` and released using `free()`. Valgrind was used to check the program for memory leaks and invalid memory operations.

Result

Memory allocation, cleanup, and Valgrind-based verification were successfully performed.

Task 4: Automated Testing

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports

To create a shell test script that automatically executes the program with test inputs and verifies the expected results.

For this task, we'll test the **Task 1 integrated calculator**.

Program

Use the same `main.c`, `calculator.c`, and `calculator.h` from Task 1.

Compile:

```
gcc main.c calculator.c -o integrated
```

Test Script

```
#!/bin/bash

echo "==== Automated Test Report ====="

echo "Test 1: Addition and Multiplication"

output=$(echo "5 4" | ./integrated)

echo "$output"

if echo "$output" | grep -q "Addition: 9" &&
  echo "$output" | grep -q "Multiplication: 20"
then
    echo "Test 1: PASS"
else
    echo "Test 1: FAIL"
fi

echo
echo "Test 2: Another Input"

output=$(echo "10 3" | ./integrated)

echo "$output"

if echo "$output" | grep -q "Addition: 13" &&
  echo "$output" | grep -q "Multiplication: 30"
then
    echo "Test 2: PASS"
else
    echo "Test 2: FAIL"
fi

echo
echo "==== Testing Completed ====="
```

Input

The test script automatically provides:

5 4

and:

10 3

Output

```
amrutha@blue:~/SKILL_Week11$ gcc main.c calculator.c -o integrated
amrutha@blue:~/SKILL_Week11$ nano test.sh
amrutha@blue:~/SKILL_Week11$ chmod +x test.sh
amrutha@blue:~/SKILL_Week11$ ./test.sh
===== Automated Test Report =====
Test 1: Addition and Multiplication
Enter two numbers: Addition: 9
Multiplication: 20
Test 1: PASS

Test 2: Another Input
Enter two numbers: Addition: 13
Multiplication: 30
Test 2: PASS

===== Testing Completed =====
```

Observation

The test script automatically supplied test inputs, captured the program output, compared it against expected values, and reported whether each test passed or failed.

Result

Automated test execution and result verification were successfully implemented.